




BENAZIR BHUTTO SHAHEED HUMAN RESOURCE RESEARCH AND DEVELOPMENT BOARD

PMU-ORIC UNIVERSITY OF SINDH, JAMSHORO.

CAPSTONE PROJECT-II

Tool Name:

CYBERGUARD

Automated Cybersecurity Assessment and Reporting Tool

Project Report:

Submitted By: Noor Zunera

Course: Cybersecurity

Submitted To: Ma'am Pareesa & Ma'am Shiza

Submission Date: 29/10/2026

1. Introduction

Cybersecurity is important for protecting computer systems, networks, and websites from potential security threats. Identifying security weaknesses helps system owners understand risks and take appropriate security measures.

CyberGuard is an automated cybersecurity assessment tool developed using Bash scripting in Kali Linux. It integrates selected security tools to perform reconnaissance, network scanning, web security assessment, and vulnerability assessment.

The project also includes a Python-based reporting module that uses the Gemini API to generate a structured HTML assessment report from collected scan results.

The main purpose of CyberGuard is to reduce repetitive manual work and organize assessment results in one place.

2. Problem Statement

Cybersecurity assessments often require multiple tools to be executed separately. Collecting their outputs, organizing findings, and preparing reports manually can be time-consuming.

CyberGuard addresses this problem by integrating selected security tools into a single Bash script. It collects their outputs and uses an AI-based reporting module to organize the available evidence into a readable HTML report.

3. Project Objectives

The main objectives of CyberGuard are:

- To automate selected cybersecurity assessment tasks.
- To gather information about a target domain or IP address.
- To identify open ports and detected services.
- To assess web technologies and potential security weaknesses.
- To collect assessment results in an organized manner.
- To generate a structured HTML report using the Gemini API.
- To reduce manual work between assessment stages.
- To present findings and possible recommendations in an understandable format.

4. Project Scope

CyberGuard is designed for authorized cybersecurity assessments of domains, IP addresses, and supported web targets.

The project covers the following stages:

1. **Reconnaissance:** Collecting domain, DNS, and available subdomain information.
2. **Network Scanning:** Identifying open ports and detected services.
3. **Web Security Assessment:** Identifying web technologies and checking for selected web security issues.
4. **Vulnerability Assessment:** Collecting potential vulnerability findings using configured security tools.
5. **Automated Reporting:** Organizing collected evidence and generating an HTML assessment report.

The tool is intended for educational purposes and authorized security testing. Its results depend on the target, available tools, network conditions, and API availability.

5. Design and Approach

CyberGuard follows a sequential workflow that collects and organizes results from different security tools.

5.1 Input

The user provides a target domain or IP address. A URL may also be required for web-related assessments. The script requests authorization confirmation before starting the assessment.

5.2 Reconnaissance

The reconnaissance stage uses tools such as WHOIS, dig, Subfinder, Sublist3r, and theHarvester to collect available information. WhatWeb helps identify technologies used by a website.

5.3 Network Scanning

Nmap is used to examine the target and identify information such as open ports and detected services. The results are saved for later review.

5.4 Web Security Assessment

Selected tools, including WhatWeb, Nikto, DIRB, and JoomScan, support web technology identification and security checks. JoomScan is used when Joomla is identified. SQLmap and PwnXSS support their respective checks when the required inputs and conditions are available.

5.5 Vulnerability Assessment

This stage collects potential security findings from the configured tools. Findings may require additional verification because automated tools can produce false positives or incomplete results.

5.6 Automated Report Generation

The Bash script combines the available results into a consolidated evidence file. A Python module then uses the Gemini API to generate a structured HTML report containing a summary, findings, severity information, potential impact, and recommendations.

The report depends on the available evidence and successful communication with the AI service.

6. Tools and Technologies Used

Tool / Technology	Purpose
Kali Linux	Cybersecurity testing environment
Bash	Main automation script
Python 3	AI-based report generation
WHOIS	Domain registration information
dig	DNS record queries
Subfinder	Subdomain discovery
Sublist3r	Subdomain discovery
theHarvester	Public information gathering
WhatWeb	Web technology identification
Nmap	Network and service scanning
Nikto	Web server security checks
DIRB	Web directory discovery
JoomScan	Joomla security assessment
SQLmap	SQL injection testing support
PwnXSS	XSS testing support
Nuclei	Template-based vulnerability checks
Gemini API	AI-assisted report generation
HTML and CSS	Structure and styling of the report

7. Implementation

CyberGuard is implemented as a Bash script that coordinates the configured assessment stages.

The script accepts target information, requests authorization confirmation, creates directories for storing results, and executes the available tools. Outputs are saved in separate files for easier review.

After the assessment stages, the script combines the available evidence into a consolidated results file.

The Python reporting module reads this evidence and uses the Gemini API to generate an HTML report. The report is designed to present assessment information in an organized and understandable format.

The Gemini API key is supplied through the `GEMINI_API_KEY` environment variable instead of being written directly into the source code. This helps reduce the risk of accidentally exposing the key.

8. Testing Process

CyberGuard should be tested in a controlled environment using systems for which permission has been obtained.

The following tests are used to evaluate the project:

1. **Input Testing:** Check whether the script accepts the intended target information.
2. **Authorization Testing:** Verify that authorization is requested before an assessment begins.
3. **Reconnaissance Testing:** Check whether available information is collected and saved.
4. **Network Scanning Testing:** Verify that Nmap results are generated and saved.
5. **Web Assessment Testing:** Check whether applicable web assessment tools run correctly.
6. **Conditional Tool Testing:** Verify that tools such as JoomScan are skipped when their required conditions are not met.
7. **Result Collection Testing:** Check whether available outputs are combined into the evidence file.
8. **AI Report Testing:** Check whether the Python module generates the expected HTML report.
9. **Error Handling Testing:** Examine the behavior when a tool is unavailable or the API returns an error.

9. Results and Output

CyberGuard is designed to perform multiple cybersecurity assessment tasks through a single automated workflow. It collects information from different security tools, saves the available results, and prepares the collected evidence for report generation.

9.1 Main Script Execution

CyberGuard starts by displaying its interface and asking the user to provide the target information. It also requests authorization confirmation before starting the assessment.

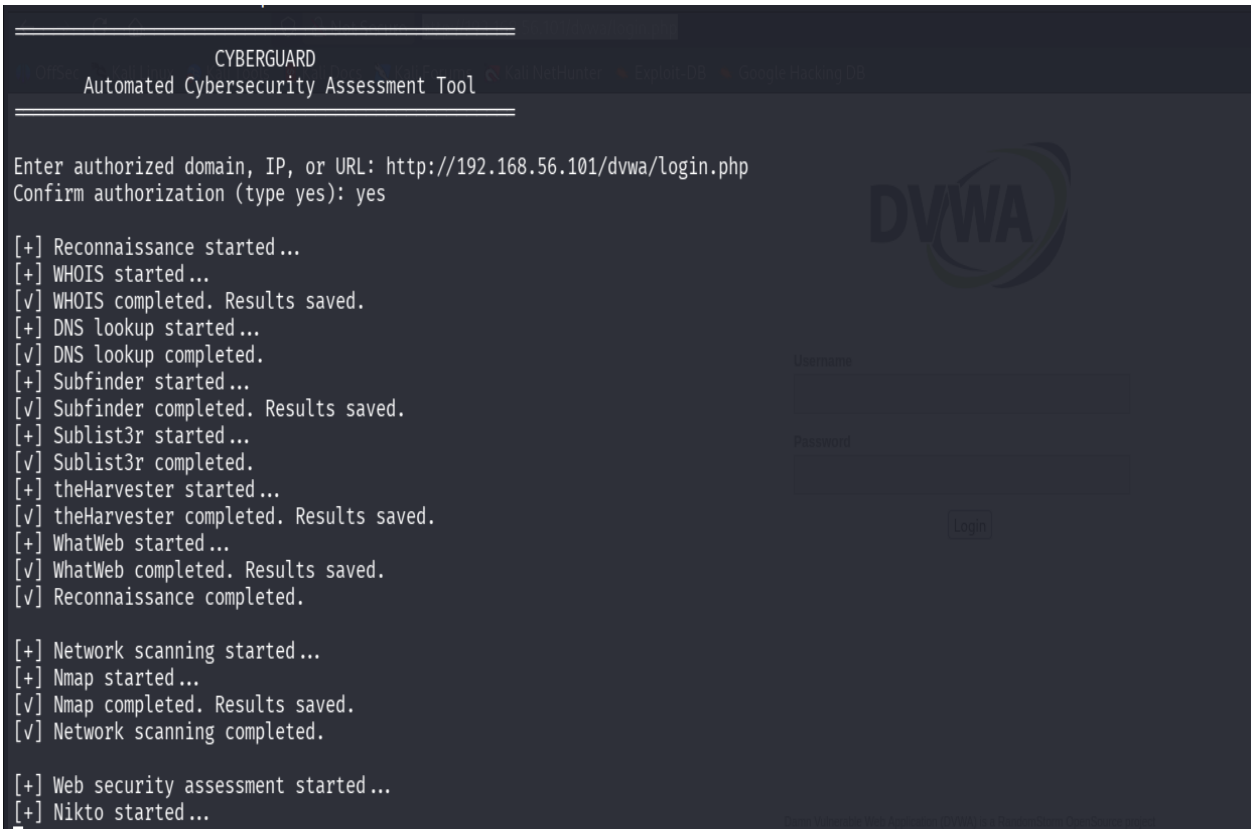


Figure 1: CyberGuard main script showing the program interface and target input.

9.2 Reconnaissance Results

The reconnaissance stage uses tools such as WHOIS and dig to collect available domain and DNS information. Other configured tools may also be used to discover subdomains and gather publicly available information.

The collected results are saved for further review and assessment.

```
#
# ARIN WHOIS data and services are subject to the Terms of Use
# available at: https://www.arin.net/resources/registry/whois/tou/
#
# If you see inaccuracies in the results, please report at
# https://www.arin.net/resources/registry/whois/inaccuracy_reporting/
#
# Copyright 1997-2026, American Registry for Internet Numbers, Ltd.
#


NetRange:       192.168.0.0 - 192.168.255.255
CIDR:           192.168.0.0/16
NetName:        PRIVATE-ADDRESS-CBLK-RFC1918-IANA-RESERVED
NetHandle:      NET-192-168-0-0-1
Parent:         NET192 (NET-192-0-0-0-0)
NetType:        IANA Special Use
OriginAS:
Organization:   Internet Assigned Numbers Authority (IANA)
RegDate:        1994-03-15
Updated:        2024-05-24
Comment:        These addresses are in use by many millions of independently operated networks, which might be as small as a single computer connected to a home gateway, and are automatica
  hundreds of millions of devices.  They are only intended for use within a private context  and traffic that needs to cross the Internet will need to use a different, unique address.
Comment:
Comment:        These addresses can be used by anyone without any need to coordinate with IANA or an Internet registry.  The traffic from these addresses does not come from ICANN or IANA.
  source of activity you may see on logs or in e-mail records.  Please refer to http://www.iana.org/abuse/answers
Comment:
Comment:        These addresses were assigned by the IETF, the organization that develops Internet protocols, in the Best Current Practice document, RFC 1918 which can be found at:
Comment:        http://datatracker.ietf.org/doc/rfc1918
Ref:            https://rdap.arin.net/registry/ip/192.168.0.0

```

Figure 2: Reconnaissance results showing collected domain or DNS information.

9.3 Network Scanning Results

CyberGuard uses Nmap to examine the target and identify information such as open ports and detected services. These results help provide an overview of the network services identified during the assessment.

The findings depend on the target and the outcome of the scan.

```
# Nmap 7.98 scan initiated Thu Oct 29 11:32:45 2026 as: /usr/lib/nmap/nmap --privileged -sV --version-light -oN network_results/nma
Nmap scan report for 192.168.56.101
Host is up (0.00087s latency).
Not shown: 977 closed tcp ports (reset)
PORT     STATE SERVICE     VERSION
21/tcp   open  ftp         vsftpd 2.3.4
22/tcp   open  ssh         OpenSSH 4.7p1 Debian 8ubuntu1 (protocol 2.0)
23/tcp   open  telnet      Linux telnetd
25/tcp   open  smtp        Postfix smtpd
53/tcp   open  domain      ISC BIND 9.4.2
80/tcp   open  http        Apache httpd 2.2.8 ((Ubuntu) DAV/2)
111/tcp  open  rpcbind
139/tcp  open  netbios-ssn Samba smbd 3.X - 4.X (workgroup: WORKGROUP)
445/tcp  open  netbios-ssn Samba smbd 3.X - 4.X (workgroup: WORKGROUP)
512/tcp  open  exec        netkit-rsh rexecd
513/tcp  open  login       OpenBSD or Solaris rlogind
514/tcp  open  shell       Netkit rshd
1099/tcp open  java-rmi    GNU Classpath grmiregistry
1524/tcp open  bindshell   Metasploitable root shell
2049/tcp open  rpcbind
2121/tcp open  ftp         ProFTPD 1.3.1
3306/tcp open  mysql       MySQL 5.0.51a-3ubuntu5
5432/tcp open  postgresql  PostgreSQL DB 8.3.0 - 8.3.7
5900/tcp open  vnc         VNC (protocol 3.3)
6000/tcp open  X11         (access denied)
6667/tcp open  irc         UnrealIRCd
8009/tcp open  ajp13       Apache Jserv (Protocol v1.3)
8180/tcp open  http        Apache Tomcat/Coyote JSP engine 1.1
MAC Address: 08:00:27:C5:E0:EA (Oracle VirtualBox virtual NIC)
Service Info: Hosts:  metasploitable.localdomain, irc.Metasploitable.LAN; OSs: Unix, Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
# Nmap done at Thu Oct 29 11:32:59 2026 -- 1 IP address (1 host up) scanned in 13.71 seconds

```

Figure 3: Network scanning output generated by Nmap.

9.4 Web Security Assessment Results

The web security assessment stage uses configured tools to identify web technologies, discover directories, and perform selected security checks where applicable. Some tools run only when their required conditions are met.

The outputs are collected so they can be reviewed and included in the overall assessment evidence.

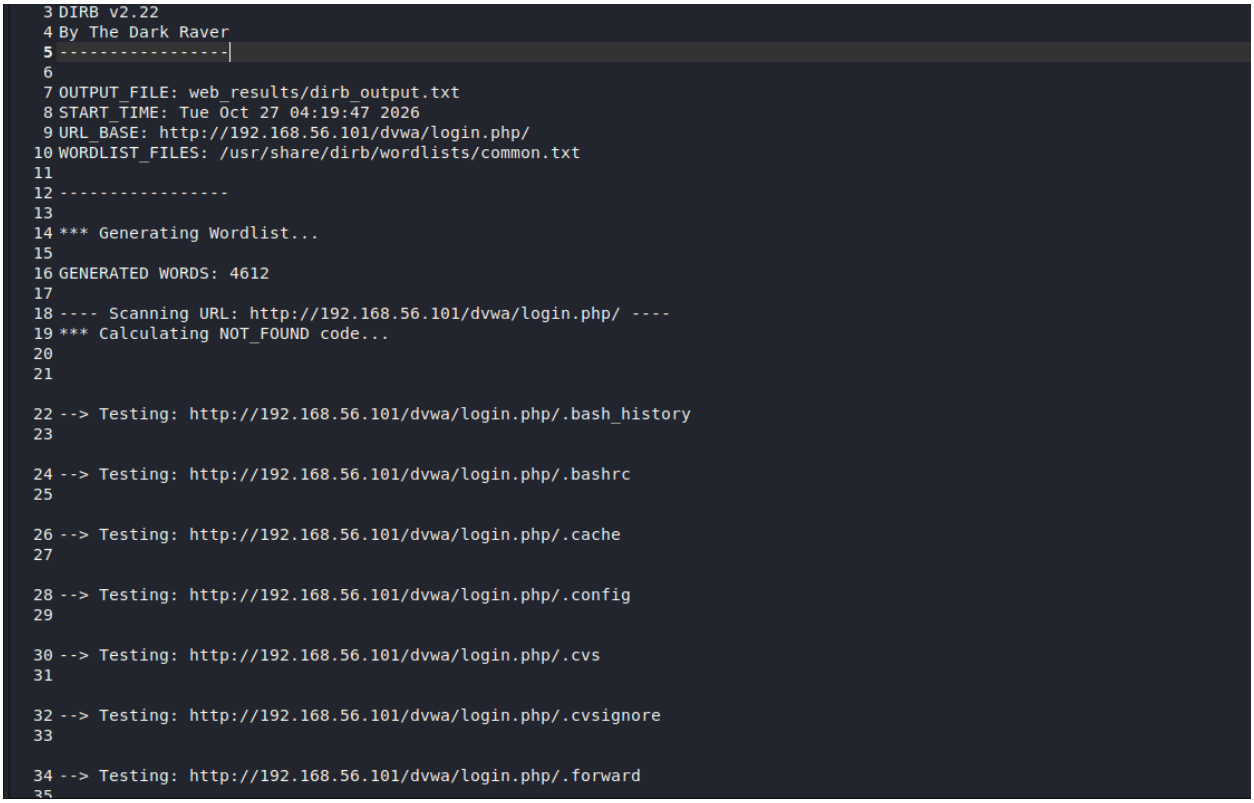


Figure 4: Output from a web security assessment tool used by CyberGuard.

9.5 Automated AI Report Generation

CyberGuard combines the available assessment evidence into a consolidated results file. Its Python reporting module uses the Gemini API to generate a structured HTML report when the API request and report-generation process succeed.

The report is designed to organize findings, potential severity, evidence, and recommendations in a readable format.



Figure 5: CyberGuard AI-generated HTML assessment report displayed in a web browser.

9.6 Saved Assessment Files

CyberGuard stores its assessment outputs in organized directories. These files allow the collected evidence to be reviewed and used during report generation.

The saved outputs may include reconnaissance information, network scan results, web assessment results, consolidated evidence, and the generated HTML report.



Figure 6: CyberGuard assessment files and generated report stored in the project directory.

9.7 Summary of Results

The project brings multiple cybersecurity assessment stages together in one Bash-based workflow. It is designed to reduce repetitive manual work by collecting outputs in one place and supporting automated HTML report generation.

The final results depend on which tools execute successfully, the information available from the target, and the availability of the Gemini API. All security findings should be reviewed before being treated as confirmed vulnerabilities.

10. Limitations

CyberGuard has some limitations:

- Results depend on the availability and configuration of the security tools.
- Some scans may take time or fail because of network conditions or target restrictions.
- Certain tools require additional inputs or specific target conditions.
- Automated scanning may produce false positives or miss security issues.
- AI-generated recommendations depend on the evidence supplied to the reporting module.
- Gemini API quota limits, authentication errors, or service interruptions may affect report generation.
- The tool does not replace manual security analysis or professional penetration testing.

11. Lessons Learned

This project provides practical experience in:

- Writing Bash scripts to automate repetitive tasks.
- Integrating multiple cybersecurity tools into one workflow.
- Managing command-line inputs and errors.
- Organizing scan outputs into files and directories.
- Using Python to process assessment evidence.
- Integrating an external API into a project.
- Generating structured HTML reports.
- Understanding the importance of verifying security findings.
- Protecting API keys and following authorized testing practices.

12. Conclusion

CyberGuard is an automated cybersecurity assessment and reporting project that integrates selected security tools into a single workflow. It covers reconnaissance, network scanning, web security assessment, vulnerability assessment, and automated reporting.

By combining Bash automation with a Python-based Gemini reporting module, the project aims to reduce repetitive manual work and present collected evidence in a structured HTML report.

The project provides practical experience in cybersecurity automation, scripting, tool integration, and report generation. The results must still be reviewed and verified to ensure their accuracy.

12. References

1. Kali Linux Documentation — https://www.kali.org/docs/
2. Nmap Reference Guide — https://nmap.org/book/man.html
3. ProjectDiscovery Documentation — https://docs.projectdiscovery.io/
4. Python Documentation — https://docs.python.org/3/
5. Google Gemini API Documentation — https://ai.google.dev/gemini-api/docs
6. WhatWeb — https://github.com/urbanadventurer/WhatWeb
7. DIRB — https://github.com/v0re/dirb
8. SQLmap — https://github.com/sqlmapproject/sqlmap